PBJ-2 Practical Assignment

Lab 3: End-to-End Hospital Appointment and Patient Triage Management

Name: Sunandha Kumar
Register No.: RA2411003010157

1. Objective

Build a hospital appointment workflow on Camunda 8 that registers a patient request, applies predefined educational triage rules, checks appointment availability through a mock scheduling API, routes cases to staff when manual intervention is needed or no slot is available, and supports monitoring and recovery in Operate. All patients are synthetic and the triage rules are an educational scenario, not clinical guidance.

2. Environment and Tools

Component	Version / details
Camunda 8 Run (c8run)	8.8.32, local: Zeebe, Operate, Tasklist, Connectors runtime
Camunda Desktop Modeler	5.49.0, target platform Camunda 8.8
Java / Spring Boot	OpenJDK 21, Spring Boot 3.5.16, camunda-spring-boot-starter 8.8.32
IDE / build	IntelliJ IDEA, Maven
Mock scheduling API	Spring REST controller in the same app, port 8092, with an outage switch

3. Architecture

- **Camunda 8 Run** executes the process, evaluates the triage DMN, runs the REST Connector and serves Tasklist and Operate.
- **Spring Boot application** (port 8092) hosts two job workers, `validate-registration` and `book-appointment`, and the mock scheduling API `GET /api/slots`.

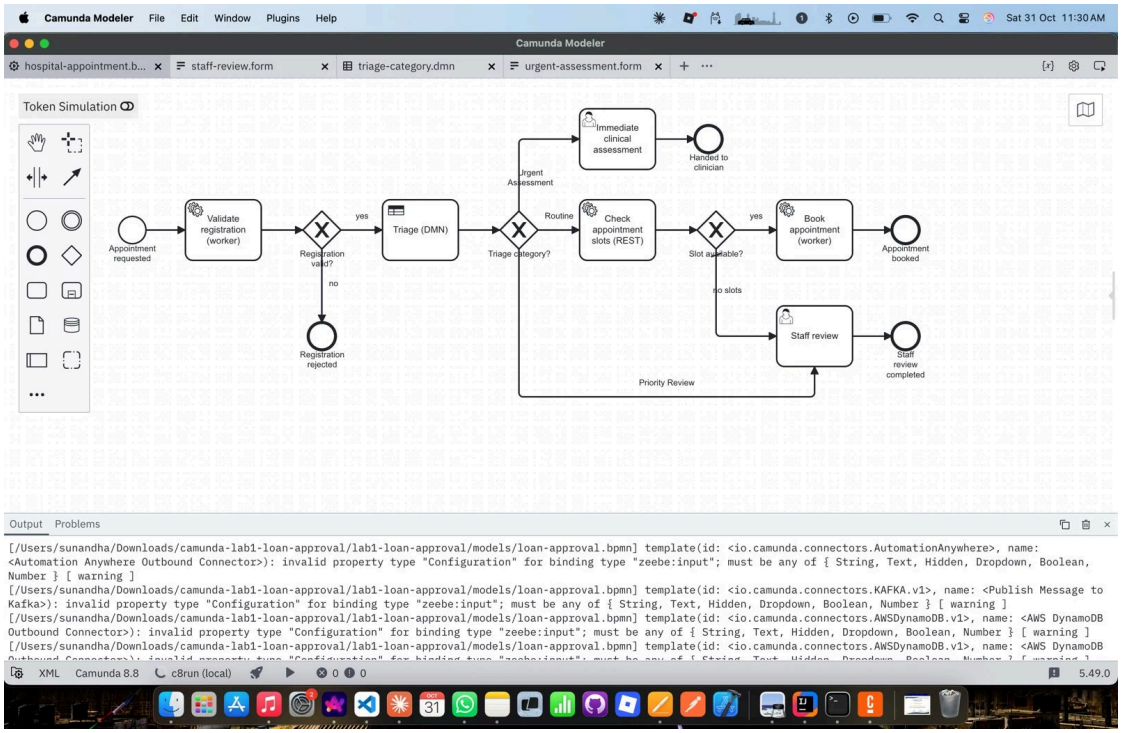


Figure 1: *Hospital Appointment and Triage BPMN process in Desktop Modeler (process id hospital-appointment)*

Execution flow: Appointment requested → Validate registration (worker) → Registration valid? → Triage (DMN) → Triage category? Urgent Assessment goes to Immediate clinical assessment (Tasklist, Critical priority) → Handed to clinician. Routine goes to Check appointment slots (REST Connector) → Slot available? → Book appointment (worker) → Appointment booked, or with no slots → Staff review. Priority Review goes directly to Staff review → Staff review completed.

4. Implementation

4.1 BPMN process

Element	Type	Configuration
Validate registration	Service task	Job type `validate-registration`, 3 retries
Registration valid?	Exclusive gateway	`registrationValid` true / false
Triage	Business rule task	DMN `triage-category`, result variable `urgencyCategory`
Triage category?	Exclusive gateway	Urgent Assessment / Routine / Priority Review
Immediate clinical assessment	User task	Assignee `demo`, form `urgent-assessment`, task priority 100 (Critical)
Check appointment slots	Service task (REST Connector)	`io.camunda:http-json:1`, GET mock scheduling API, 3 retries
Slot available?	Exclusive gateway	`slotCount > 0`; default → Staff review
Book appointment	Service task	Job type `book-appointment` (books the first slot)
Staff review	User task	Assignee `demo`, form `staff-review`

4.2 DMN triage decision table

Decision **`triage-category`**, hit policy **FIRST**. Inputs are the predefined urgency indicators **`chestPain`**, **`breathingDifficulty`**, **`painScore`** (0–10) and **`symptomDays`**. Output: **`urgencyCategory`**.

Rule	chestPain	breathingDifficulty	painScore	symptomDays	urgencyCategory
R1	true	–	–	–	Urgent Assessment
R2	–	true	–	–	Urgent Assessment
R3	–	–	>= 7	–	Priority Review
R4	–	–	–	> 14	Priority Review
R5	–	–	–	–	Routine

Educational rationale: red-flag indicators are checked first, then severity (pain) and duration, and everything else is routine. Sample evaluations from the runs: PT-1003 (chest pain) → R1 Urgent Assessment; PT-1002 (pain 8) → R3 Priority Review; PT-1001, PT-1004 and PT-1005 → R5 Routine.

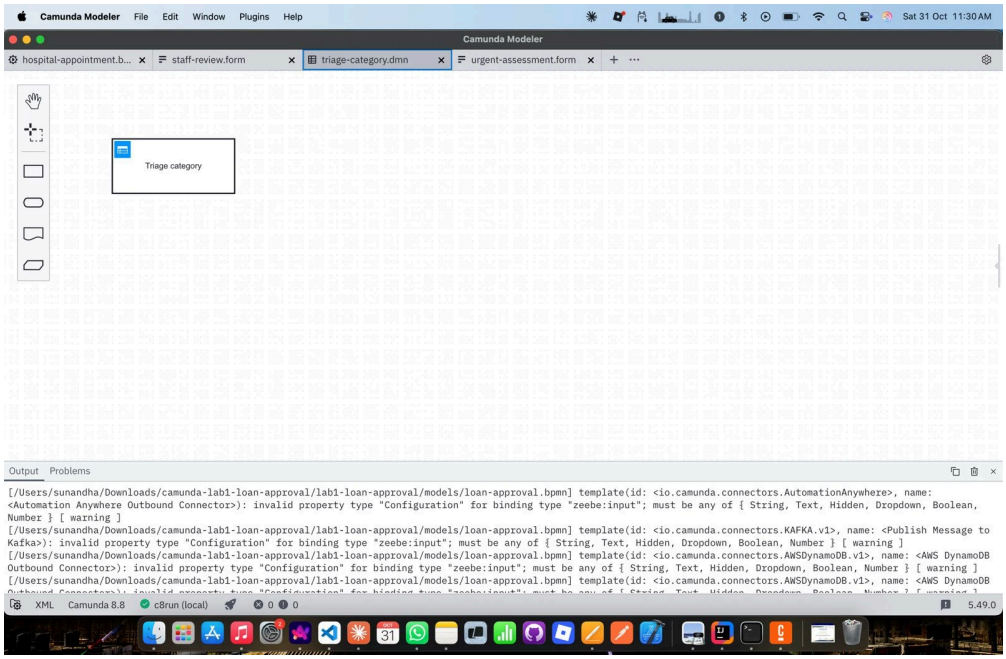


Figure 2: DMN decision requirements diagram (triage-category) in Desktop Modeler

4.3 Spring Boot job workers

- **validate-registration** checks that `patientRef` matches the synthetic PT-1234 format, that `symptomsSummary` is present, that `preferredDate` is a valid ISO date, that `preferredSession` is MORNING or AFTERNOON, that both urgency booleans are present, that `painScore` is 0–10 and that `symptomDays` is 0 or more. It returns `registrationValid`, `validationErrors` and prepares the availability request by setting `department = "GP"`.
- **book-appointment** takes the first entry of `availableSlots` and returns `bookingStatus = "BOOKED"`, `bookedSlot` and a `bookingRef` such as BK-1001-0900.

Configuration: `server.port=8092`, `camunda.client.grpc-address=http://localhost:26500`, `camunda.client.rest-address=http://localhost:8080`. Three JUnit tests cover a valid registration, invalid date/pain values and the booking reference (`mvn test`: 3 passed).

4.4 REST Connector and mock scheduling API

Setting	Value
Method / auth	GET, no authentication
URL	`="http://localhost:8092/api/slots?department=" + department + "&date=" + preferredDate + "&session=" + preferredSession`
Result expression	`={availableSlots: response.body.slots, slotCount: count(response.body.slots)}`
Retries	3, backoff PT5S; HTTP errors raise an incident after the last retry

Mock behaviour: MORNING returns 09:00, 09:30 and 10:00; AFTERNOON returns 14:00 and 14:30. 2026-11-06 is fully booked and returns an empty list. When the outage switch is on (**`POST /mock/scheduling/outage?down=true`**), every call returns HTTP 503.

`GET /api/slots?department=GP&date=2026-11-03&session=MORNING`

`200 {"department":"GP","date":"2026-11-03","session":"MORNING","slots":["09:00","09:30","10:00"]}`

`GET /api/slots?department=GP&date=2026-11-06&session=MORNING`

`200 {"department":"GP","date":"2026-11-06","session":"MORNING","slots":[]}`

`(outage on)  503 {"error":"Scheduling service unavailable"}`



Figure 3: Mock scheduling API called from the terminal

4.5 Human tasks (Tasklist)

- **Immediate clinical assessment** (form `urgent-assessment`, Critical priority) shows the patient reference, symptoms and red-flag indicators, and records a simulated `assessmentOutcome` (seen by on-call clinician / escalated to emergency department) plus notes.
- **Staff review** (form `staff-review`) is used for Priority Review cases and for routine cases with no suitable slot. It shows the symptoms, pain, triage category, preferred appointment and number of slots found, and records the `bookingStatus` outcome (booked manually / waitlisted / referred) plus staff notes.

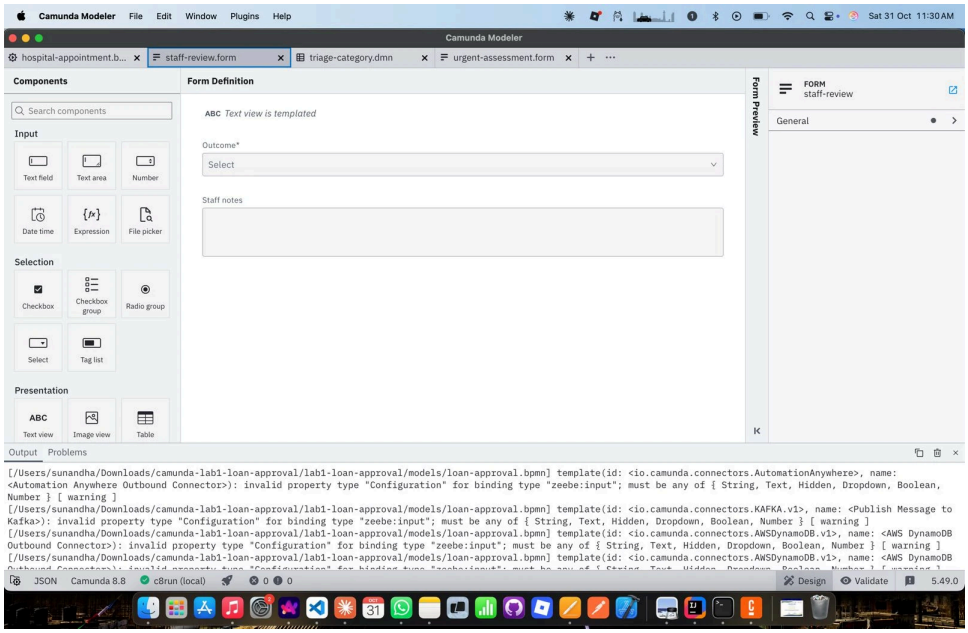


Figure 4: *Staff review form in Desktop Modeler*

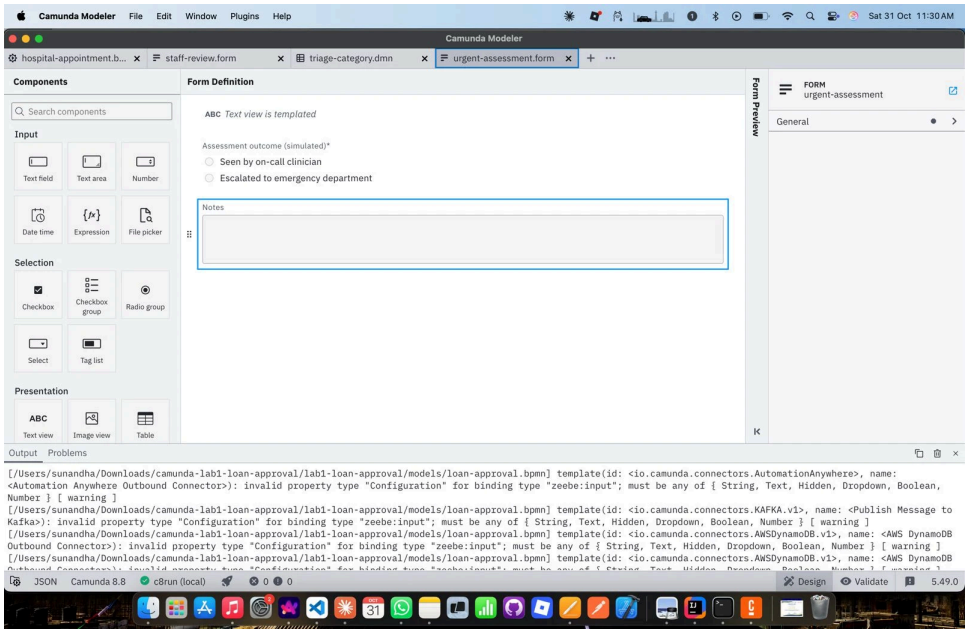


Figure 5: *Urgent assessment form in Desktop Modeler*

4.6 Process-variable dictionary and routing

Variable	Type	Set by	Purpose
patientRef	String	Start	Synthetic patient reference
preferredDate / preferredSession	String	Start	Appointment preference
symptomsSummary	String	Start	Synthetic symptom description
chestPain / breathingDifficulty	Boolean	Start	Urgency indicators (DMN inputs)
painScore / symptomDays	Number	Start	Urgency indicators (DMN inputs)
registrationValid / validationErrors	Boolean / List	Worker	Validation result
department	String	Worker	Clinic used in the availability request
urgencyCategory	String	DMN	Routine / Priority Review / Urgent Assessment
availableSlots / slotCount	List / Number	REST Connector	Slots returned by scheduling
bookingStatus	String	Booking worker or staff form	BOOKED / BOOKED_MANUALLY / WAITLISTED / REFERRED
bookedSlot / bookingRef	String	Booking worker	Confirmed slot and reference
assessmentOutcome	String	Urgent form	Simulated assessment result
staffNotes / clinicianNotes	String	Forms	Free-text notes

Gateway	Condition	Route
Registration valid?	`registrationValid = true / false`	Triage / Registration rejected
Triage category?	`urgencyCategory = "Urgent Assessment"`	Immediate clinical assessment
”	`urgencyCategory = "Routine"`	Check appointment slots
”	`urgencyCategory = "Priority Review"`	Staff review
Slot available?	`slotCount > 0`	Book appointment
”	default	Staff review (no suitable slot)

5. Deployment and Execution

The DMN, both forms and the BPMN were deployed from Desktop Modeler to the local cluster (process version 2). The Spring Boot app was started with **`mvn spring-boot:run`**, and each scenario was started through the Orchestration Cluster REST API:

```
curl -u demo:demo -H "Content-Type: application/json" \

  -X POST http://localhost:8080/v2/process-instances -d @test-data/1-routine-booked.json
```

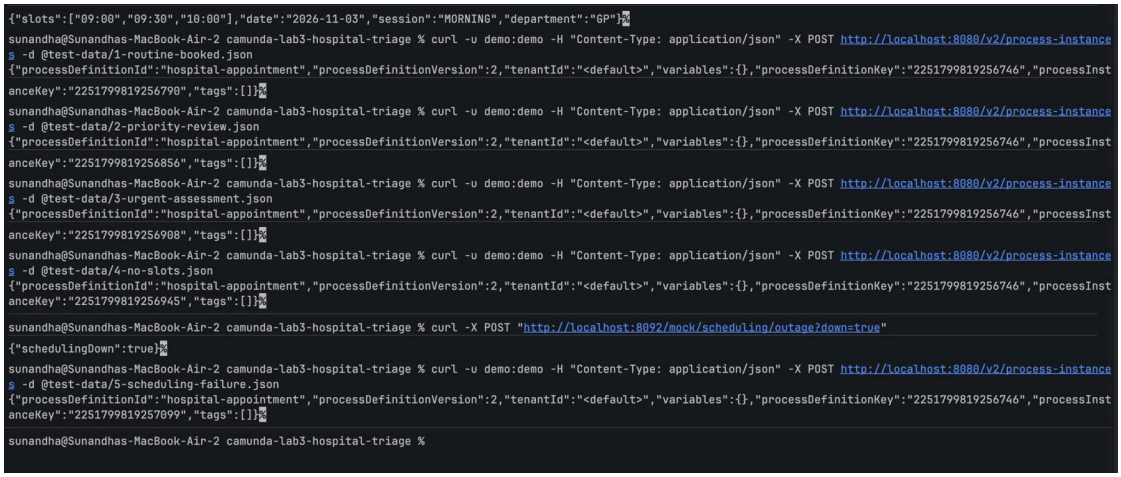


Figure 6: Starting the five scenario instances and switching the scheduling outage on

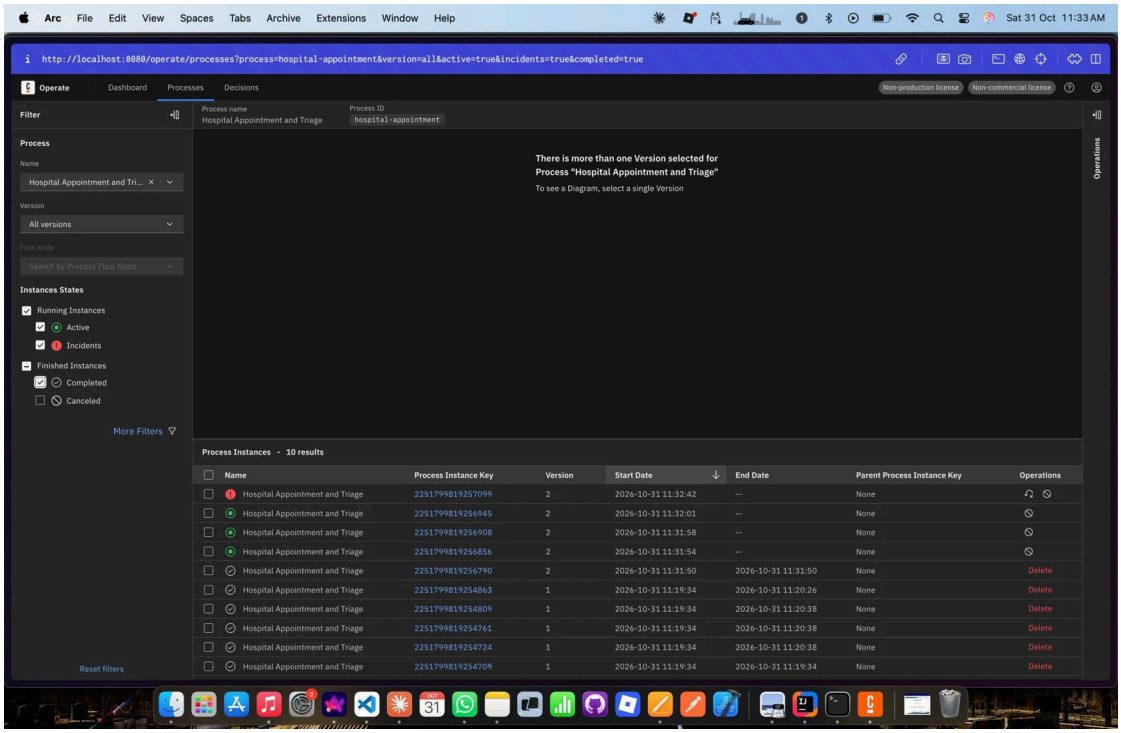


Figure 7: Operate: Hospital Appointment and Triage instances (version 2 = test runs for this report)

6. Test Results

All five required scenarios behaved as expected.

#	Scenario	Instance	Actual outcome	Status
1	Routine with available slot (PT-1001)	…256790	Routine, 3 slots → booked 09:00 → Appointment booked	Pass
2	Priority staff review (PT-1002)	…256856	Priority Review → Staff review → BOOKED_MANUALLY	Pass
3	Urgent assessment (PT-1003)	…256908	Urgent Assessment → Critical task → SEEN_BY_CLINICIAN → Handed to clinician	Pass
4	No appointment slots (PT-1004)	…256945	Routine, 0 slots → Staff review → WAITLISTED	Pass
5	Scheduling API failure + recovery (PT-1005)	…257099	503 incident → API restored → Retry → booked 14:00	Pass

6.1 Scenario 1: Routine request with an available slot

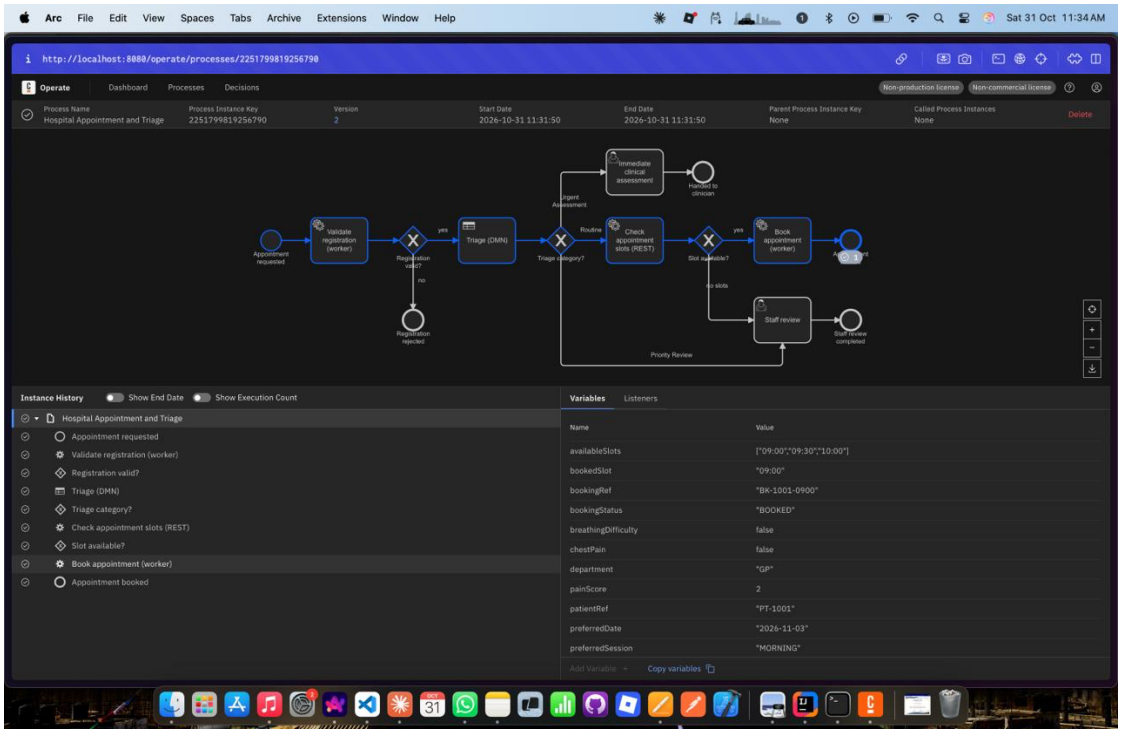


Figure 8: PT-1001: Routine → 3 slots → booked 09:00 (BK-1001-0900) → Appointment booked

6.2 Scenario 2: Request routed for priority staff review

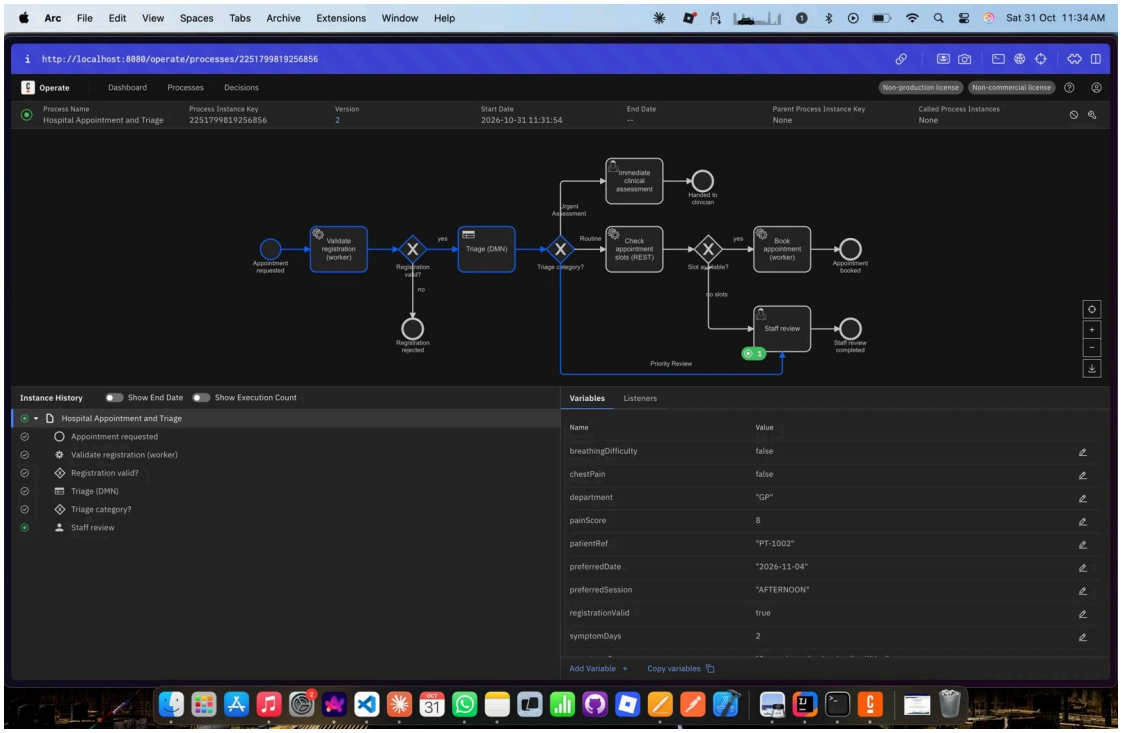


Figure 9: PT-1002: pain score 8 → Priority Review → waiting at Staff review

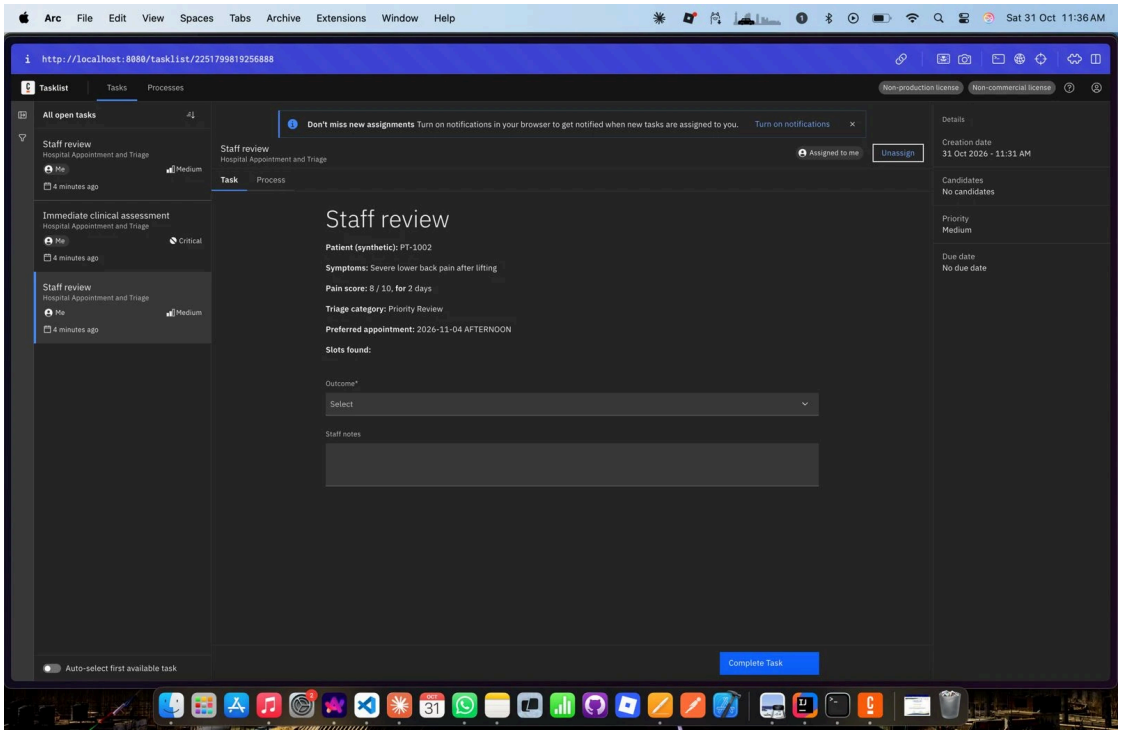


Figure 10: PT-1002: staff review form in Tasklist

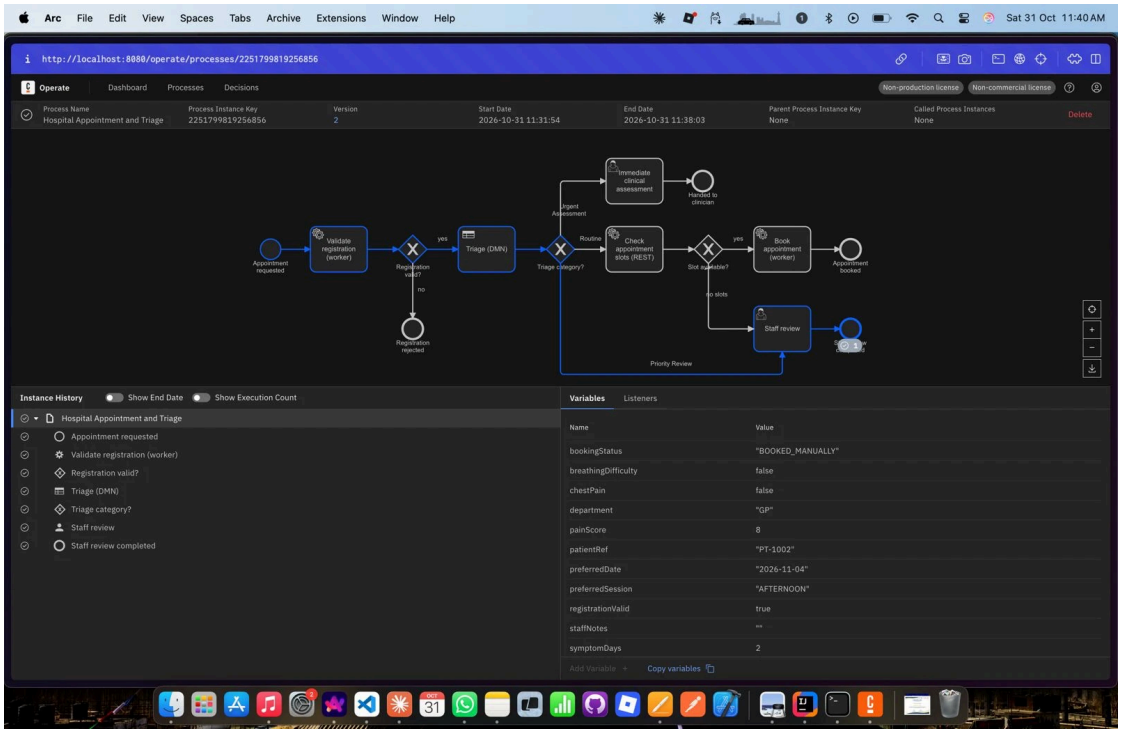


Figure 11: PT-1002: BOOKED_MANUALLY → Staff review completed

6.3 Scenario 3: Urgent assessment routed to immediate human assessment

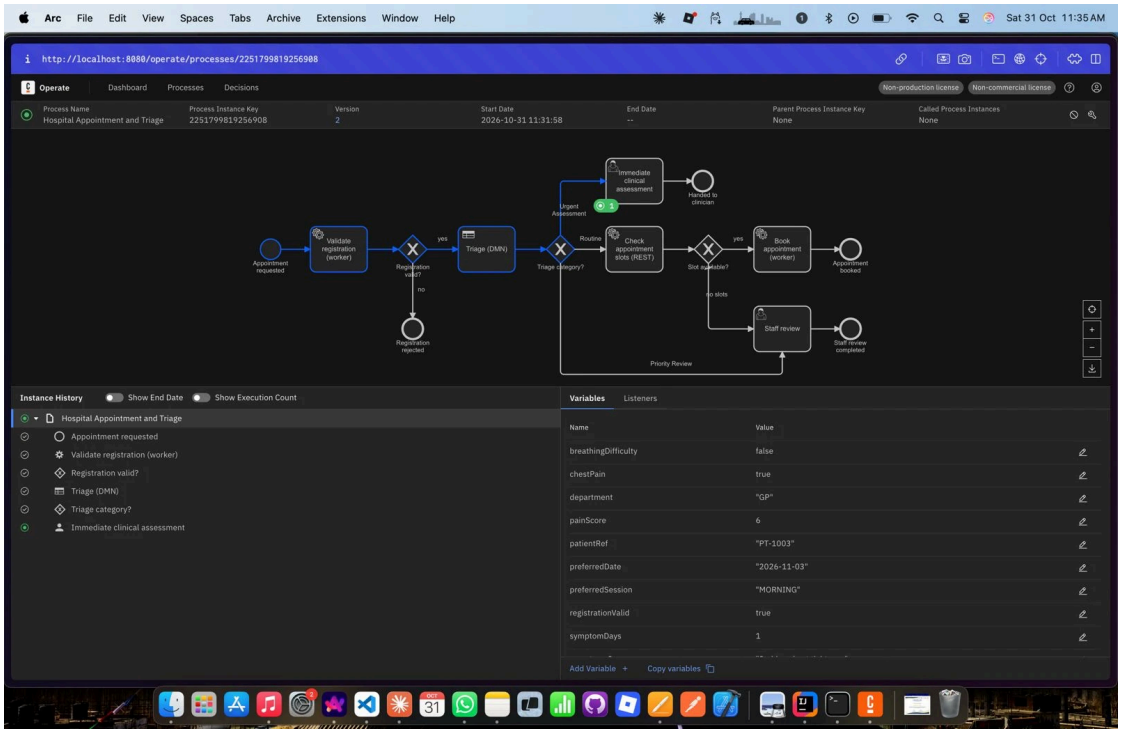


Figure 12: PT-1003: chest pain → Urgent Assessment → waiting at Immediate clinical assessment

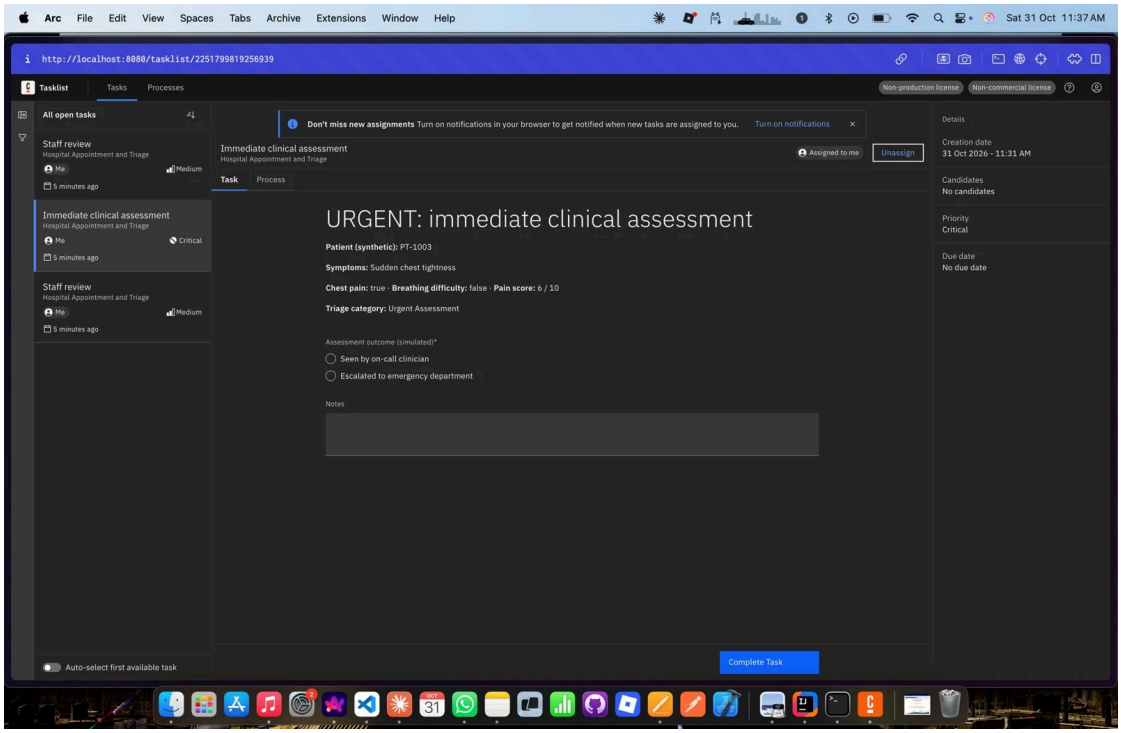


Figure 13: PT-1003: urgent assessment form in Tasklist (priority Critical)

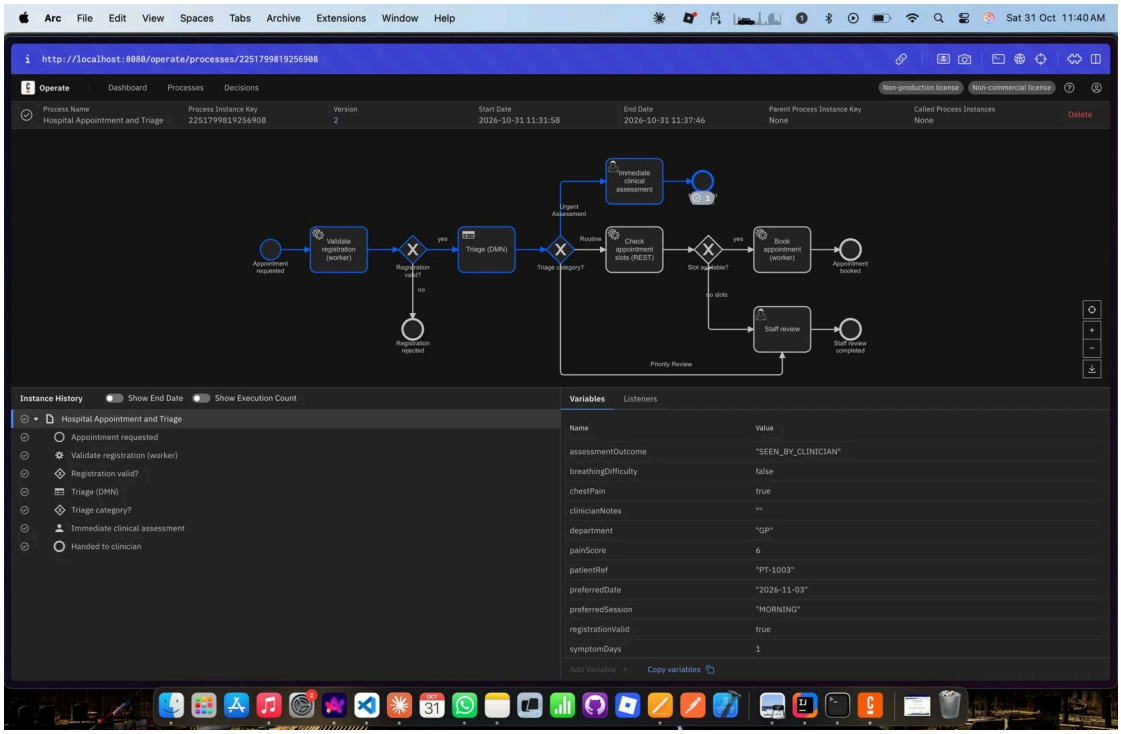


Figure 14: PT-1003: SEEN_BY_CLINICIAN → Handed to clinician

6.4 Scenario 4: No appointment slots available

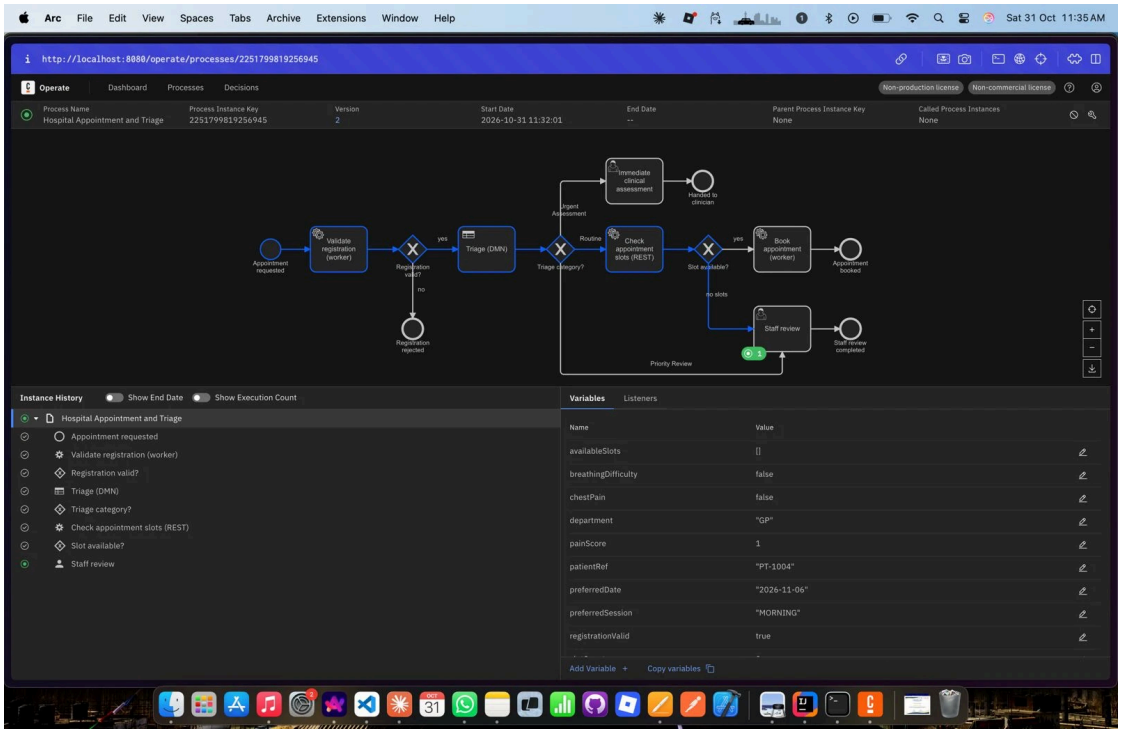


Figure 15: PT-1004: fully booked date → slotCount 0 → Staff review

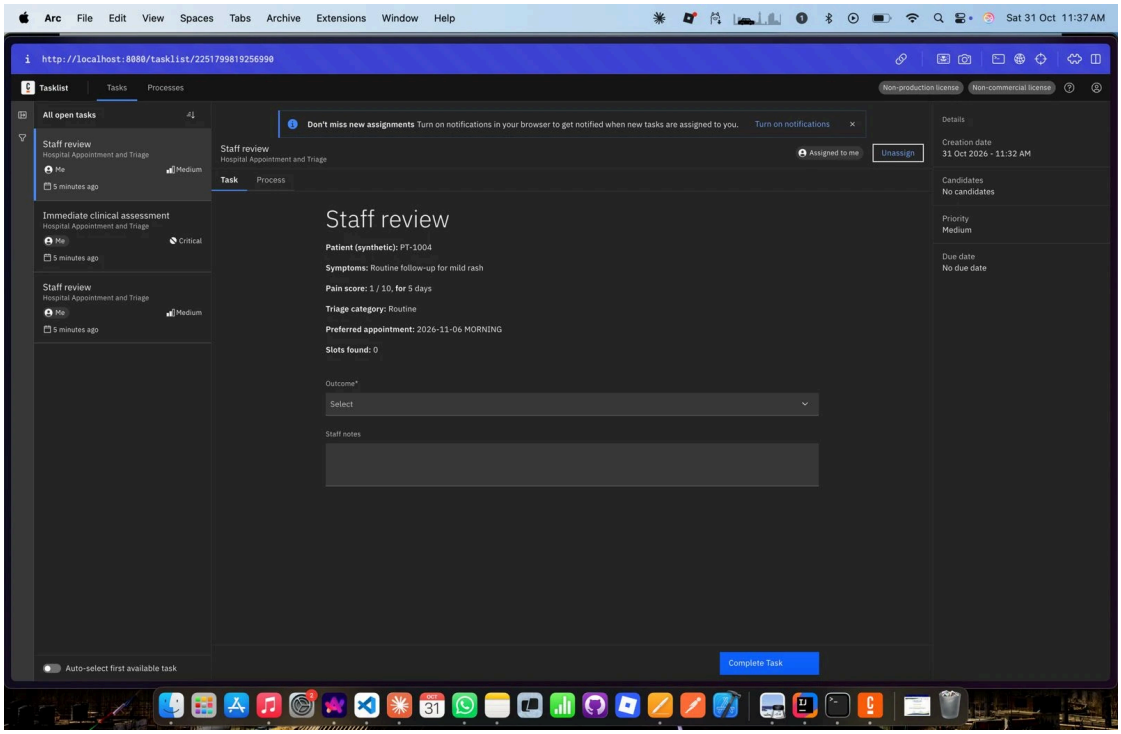


Figure 16: PT-1004: staff review form showing Slots found: 0

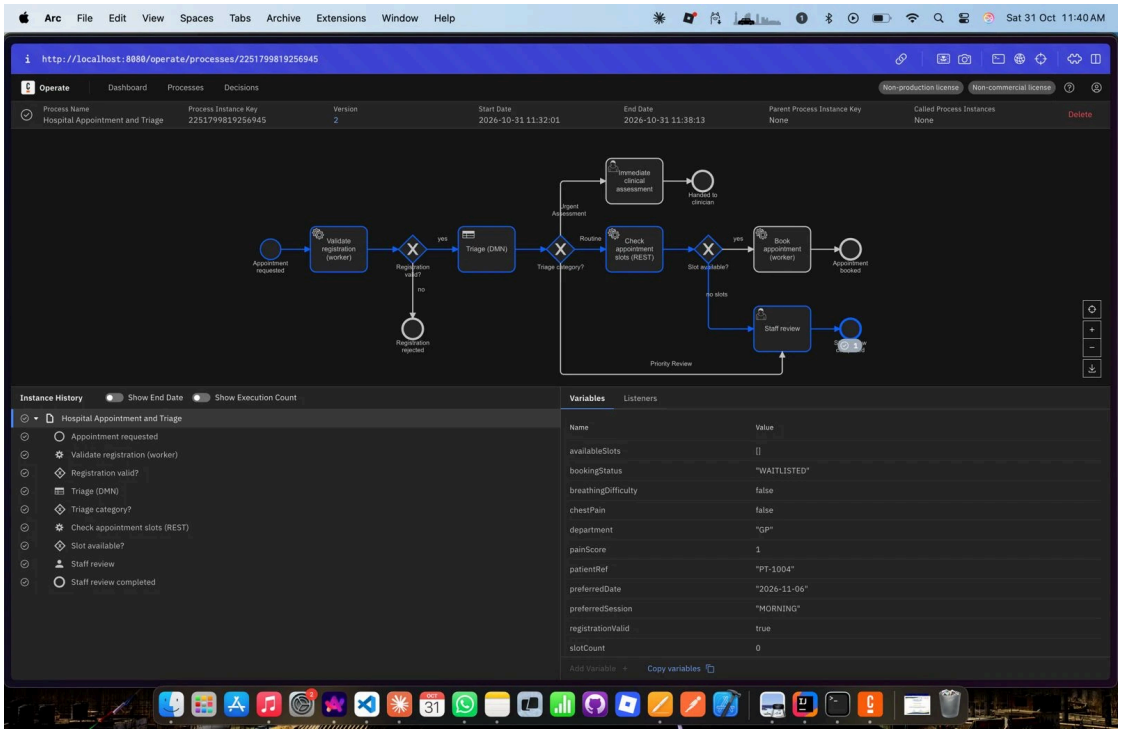


Figure 17: PT-1004: WAITLISTED → Staff review completed

7. Incident Investigation and Recovery

Scenario 5 simulated a scheduling-service failure.

1. **Simulate:** `POST /mock/scheduling/outage?down=true`, then start request PT-1005.
2. **Detect:** Operate shows an incident on Check appointment slots after the connector's three retries (No more retries left).
3. **Diagnose:** the incident's error variables show a `ConnectorException` with HTTP code 503 and the body `Scheduling service unavailable`; the input mappings show the called URL.
4. **Correct:** `POST /mock/scheduling/outage?down=false` restored the scheduling service.
5. **Retry:** the incident was retried from Operate. The connector retrieved two afternoon slots, the booking worker booked 14:00 (BK-1005-1400), and the request ended at Appointment booked at 11:41:50.

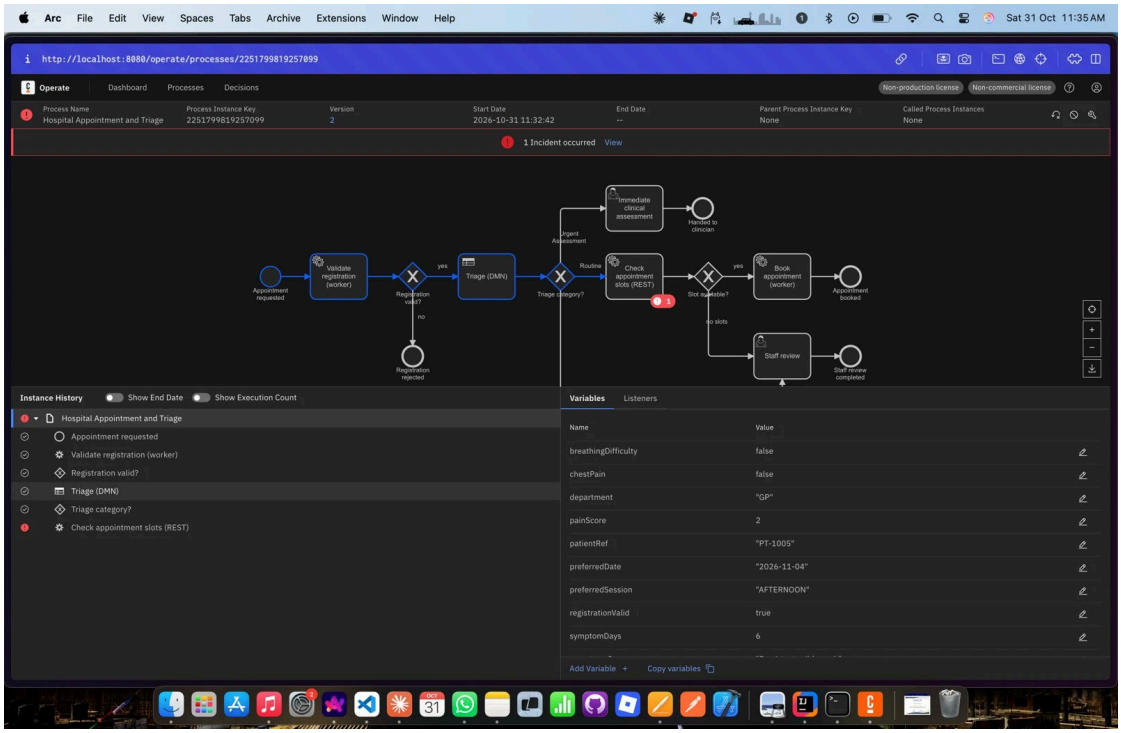


Figure 18: *PT-1005: incident on Check appointment slots*

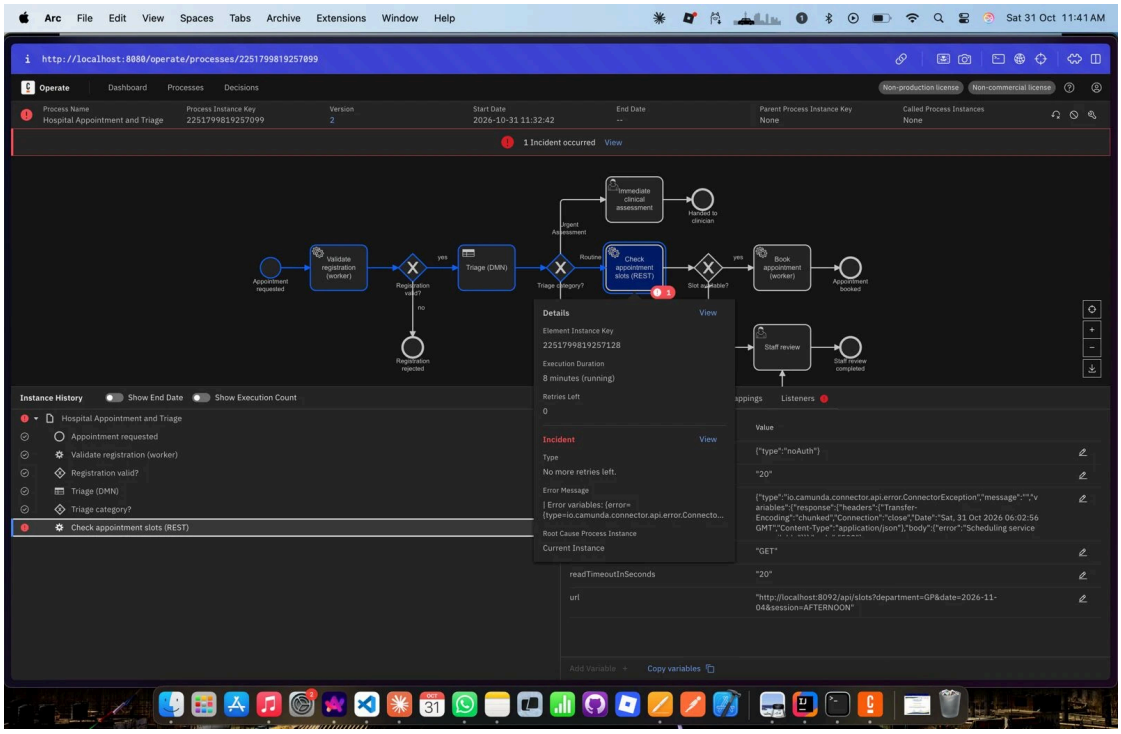


Figure 19: *PT-1005: incident details, HTTP 503 from the scheduling API, 0 retries left*

```
sunandha@Sunandhas-MacBook-Air-2 camunda-lab3-hospital-triage % curl -X POST "http://localhost:8092/mock/scheduling/outage?down=false"
{"schedulingDown":false}%
```

Figure 20: *Corrective action: scheduling outage switched off*

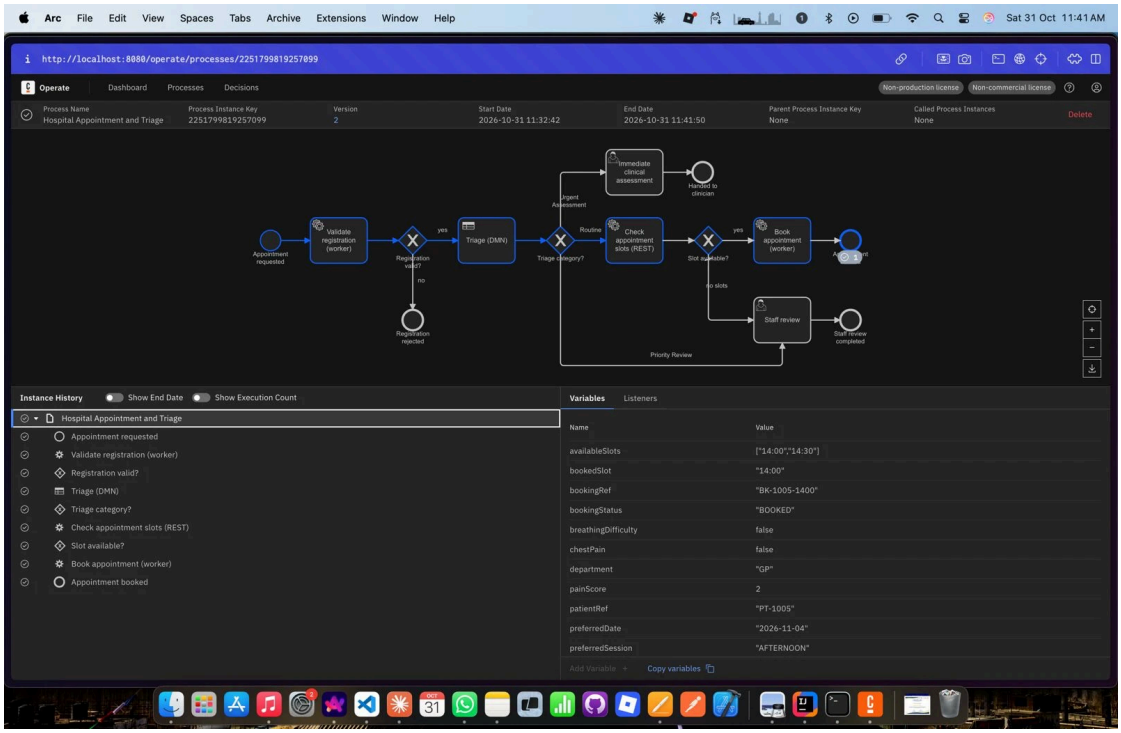


Figure 21: *PT-1005: after Retry → booked 14:00 → Appointment booked*

8. Assumptions and Limitations

- Triage rules are predefined for the educational scenario and are not clinically validated; all patients are synthetic.
- All requests go to a single department (GP); slots are static and are not reserved after booking.
- Human tasks are assigned to the single `demo` user; the urgent path only simulates handing over to a clinician.
- The scheduling API is a mock without authentication; scheduling errors are recovered through Operate rather than an alternative path.

9. Conclusion

The hospital appointment and triage workflow was modelled, deployed and executed end to end on Camunda 8. A Java worker validated registrations and prepared the availability request, DMN classified requests as Routine, Priority Review or Urgent Assessment, gateways routed cases by triage category and slot availability, a REST Connector retrieved slots from the mock scheduling API, a second worker booked appointments, and Tasklist forms supported urgent assessment and staff review. A simulated scheduling outage was detected, diagnosed and recovered in Operate. All five required scenarios passed.

Appendix: How to Run

1. Start Camunda: `cd c8run-8.8.32 && ./start.sh`
2. Start the workers and mock API: `mvn spring-boot:run` (port 8092).

3. Deploy `triage-category.dmn`, `staff-review.form`, `urgent-assessment.form` and `hospital-appointment.bpmn` from Desktop Modeler.
4. Start scenarios with the curl command in section 5 using the files in `test-data/`.
5. For scenario 5, wait about 10 seconds after scenario 4, switch the outage on, start the request, then switch it off before retrying.